

התאמת מחרוזות

חזרה קצרה | תרגילים

התאמת מחרוזות

הגדרת הבעיה

נתון טקסט T במערך $T[1...n]$ שגודלו n ונתונה תבנית P במערך $P[1...m]$ שגודלה m .
אומרים ש- P מופיעה ב- T בהזזה של s אם $0 \leq s \leq n - m$ וכן $P[1...m] = T[s+1...s+m]$.

אם P מופיעה ב- T בהזזה s , אז s נקראת הזזה חוקית (valid shift), אחרת s היא הזזה לא חוקית.

המטרה

למצוא את כל ההזזות החוקיות של P בתוך T .

התאמת מחרוזות – האלגוריתם הנאיבי

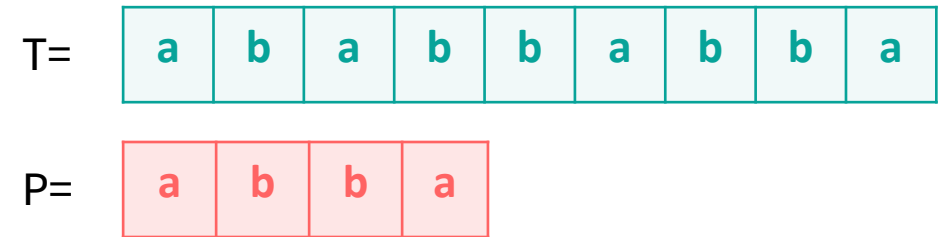
Naive-String-Matcher(T,P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. **for** $s \leftarrow 0$ **to** $n-m$
4. **do if** $P[1\dots m] = T[s+1\dots s+m]$
5. **then** print "Pattern occurs with shift" s

התאמת מחרוזות – האלגוריתם הנאיבי

Naive-String-Matcher(T,P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. **for** $s \leftarrow 0$ **to** $n-m$
4. **do if** $P[1\dots m] = T[s+1\dots s+m]$
5. **then** print "Pattern occurs with shift" s



התאמת מחרוזות – האלגוריתם הנאיבי

Naive-String-Matcher(T,P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. **for** $s \leftarrow 0$ **to** $n-m$
4. **do if** $P[1\dots m] = T[s+1\dots s+m]$
5. **then** print "Pattern occurs with shift" s

T=

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

P=

a	b	b	a
---	---	---	---

$n=9$

$m=4$

$s=0$

התאמת מחרוזות – האלגוריתם הנאיבי

Naive-String-Matcher(T,P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. **for** $s \leftarrow 0$ **to** $n-m$
4. **do if** $P[1\dots m] = T[s+1\dots s+m]$
5. **then** print "Pattern occurs with shift" s

T=

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

P=

a	b	b	a
---	---	---	---

$n=9$

$m=4$

$s=0$

התאמת מחרוזות – האלגוריתם הנאיבי

Naive-String-Matcher(T,P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. **for** $s \leftarrow 0$ **to** $n-m$
4. **do if** $P[1\dots m] = T[s+1\dots s+m]$
5. **then** print "Pattern occurs with shift" s

T=

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

P=

a	b	b	a
---	---	---	---

$n=9$

$m=4$

$s=1$

התאמת מחרוזות – האלגוריתם הנאיבי

Naive-String-Matcher(T,P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. **for** $s \leftarrow 0$ **to** $n-m$
4. **do if** $P[1\dots m] = T[s+1\dots s+m]$
5. **then** print "Pattern occurs with shift" s

T=

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

P=

a	b	b	a
---	---	---	---

$n=9$

$m=4$

$s=1$

התאמת מחרוזות – האלגוריתם הנאיבי

Naive-String-Matcher(T,P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. **for** $s \leftarrow 0$ **to** $n-m$
4. **do if** $P[1\dots m] = T[s+1\dots s+m]$
5. **then** print "Pattern occurs with shift" s

T=

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

P=

a	b	b	a
---	---	---	---

$n=9$

$m=4$

$s=2$

התאמת מחרוזות – האלגוריתם הנאיבי

Naive-String-Matcher(T,P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. **for** $s \leftarrow 0$ **to** $n-m$
4. **do if** $P[1\dots m] = T[s+1\dots s+m]$
5. **then** print "Pattern occurs with shift" s

T=

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

P=

a	b	b	a
---	---	---	---

$n=9$

$m=4$

$s=2$

התאמת מחרוזות – האלגוריתם הנאיבי

Naive-String-Matcher(T,P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. **for** $s \leftarrow 0$ **to** $n-m$
4. **do if** $P[1\dots m] = T[s+1\dots s+m]$
5. **then print** "Pattern occurs with shift" s

- Pattern occurs with shift 2

T=

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

P=

a	b	b	a
---	---	---	---

$n=9$

$m=4$

$s=2$

התאמת מחרוזות – האלגוריתם הנאיבי

Naive-String-Matcher(T,P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. **for** $s \leftarrow 0$ **to** $n-m$
4. **do if** $P[1\dots m] = T[s+1\dots s+m]$
5. **then** print "Pattern occurs with shift" s

- Pattern occurs with shift 2

T=

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

P=

a	b	b	a
---	---	---	---

$n=9$

$m=4$

$s=3$

התאמת מחרוזות – האלגוריתם הנאיבי

Naive-String-Matcher(T,P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. **for** $s \leftarrow 0$ **to** $n-m$
4. **do if** $P[1\dots m] = T[s+1\dots s+m]$
5. **then** print "Pattern occurs with shift" s

- Pattern occurs with shift 2

T=

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

P=

a	b	b	a
---	---	---	---

$n=9$

$m=4$

$s=4$

התאמת מחרוזות – האלגוריתם הנאיבי

Naive-String-Matcher(T,P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. **for** $s \leftarrow 0$ **to** $n-m$
4. **do if** $P[1\dots m] = T[s+1\dots s+m]$
5. **then** print "Pattern occurs with shift" s

- Pattern occurs with shift 2

T=

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

P=

a	b	b	a
---	---	---	---

$n=9$

$m=4$

$s=5$

התאמת מחרוזות – האלגוריתם הנאיבי

Naive-String-Matcher(T,P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. **for** $s \leftarrow 0$ **to** $n-m$
4. **do if** $P[1\dots m] = T[s+1\dots s+m]$
5. **then print** "Pattern occurs with shift" s

- Pattern occurs with shift 2
- Pattern occurs with shift 5

T=

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

P=

a	b	b	a
---	---	---	---

$n=9$

$m=4$

$s=5$

התאמת מחרוזות – האלגוריתם הנאיבי

Naive-String-Matcher(T,P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. **for** $s \leftarrow 0$ **to** $n-m$
4. **do if** $P[1\dots m] = T[s+1\dots s+m]$
5. **then** print "Pattern occurs with shift" s

- Pattern occurs with shift 2
- Pattern occurs with shift 5

T=

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

P=

a	b	b	a
---	---	---	---

$n=9$

$m=4$

$s=5$

התאמת מחרוזות – האלגוריתם הנאיבי

Naive-String-Matcher(T,P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. **for** $s \leftarrow 0$ **to** $n-m$
4. **do if** $P[1\dots m] = T[s+1\dots s+m]$
5. **then** print "Pattern occurs with shift" s

$\Theta(nm)$

- Pattern occurs with shift 2
- Pattern occurs with shift 5

T=

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

P=

a	b	b	a
---	---	---	---

$n=9$

$m=4$

$s=5$

התאמת מחרוזות – אלגוריתם רבין-קרפ

הרעיון:

ידוע שהשוואת מספרים נחשבת פעולה זולה

ולכן נמיר את התווים לספרות.

עבור כל m ספרות רצופות נבצע המרה למספר מבסיס d

כדי להימנע מ-overflow – במהלך ההמרה לבסיס d נבצע modulo

בכל שלב.

התמודדות עם התנגשויות:

מכיוון שייתכנו התנגשויות (שני מספרים שונים שערך הסולו

שלהם זהה) – בכל התאמה נחזור לתבנית המקורית ונשווה את

האותיות.

עלות:

$$\Theta(n) + O\left(m\left(V + \frac{n}{q}\right)\right)$$

m - עלות השוואת האותיות
 V - מספר ההזזות החוקיות
 $\frac{n}{q}$ - מספר ההזזות הלא חוקיות

Rabin-Karp-Matcher(T,P,d,q)

1. $n \leftarrow \text{length}[T]$

2. $m \leftarrow \text{length}[P]$

3. $h \leftarrow d^{m-1} \bmod q$

4. $p \leftarrow 0$

5. $t_0 \leftarrow 0$

6. **for** $i \leftarrow 1$ **to** m

7. **do** $p \leftarrow (dp + P[i]) \bmod q$

8. $t_0 \leftarrow (dt_0 + T[i]) \bmod q$

9. **for** $s \leftarrow 0$ **to** $n-m$

10. **do if** $p = t_s$

11. **then if** $P[1\dots m] = T[s+1\dots s+m]$

12. **then** print "Pattern occurs with shift" s

13. **if** $s < n-m$

14. **then** $t_{s+1} \leftarrow (d(t_s - T[s+1]h) + T[s+m+1]) \bmod q$

תרגיל 1

נתון טקסט T ושתי תבניות P_1, P_2 .

כתבו אלגוריתם המוצא את כל מופעי $P_1 * P_2$ כאשר ה- $*$ היא מחרוזת תווים באורך כלשהו.

עליכם להדפיס למסך את ההיסט ואת אורך המחרוזת $P_1 * P_2$.

ה- $*$ משמשת כ-wildcard לתבנית טקסט.

תרגיל 1

נתון טקסט T ושתי תבניות P_1, P_2 .

כתבו אלגוריתם המוצא את כל מופעי $P_1 * P_2$ כאשר ה- $*$ היא מחרוזת תווים באורך כלשהו.

עליכם להדפיס למסך את ההיסט ואת אורך המחרוזת $P_1 * P_2$.

ה- $*$ משמשת כ-wildcard לתבנית טקסט.

דוגמה

עבור הקלט הבא:

$P_1=aa$ $P_2=bb$ $T=aaabbb$ $m_1=2$ $m_2=2$ $n=6$

הפלט של האלגוריתם יהיה:

aaabbb $S=0$ $L=5$

aaabbb $S=0$ $L=6$

aaabbb $S=1$ $L=4$

aaabbb $S=1$ $L=5$

פתרון

נריץ רבין-קארפ פעם אחת עם P1 ופעם שניה עם P2

ונדפיס את הפתרונות שעבורם האינדקס של הפתרון עם P2 יהיה לפחות m_1 יותר מהאינדקס של הפתרון עם P1:

Find-Wildcard-String(T, P1, P2, m_1 , m_2 , d, q)

1. valid-p1-shifts $\leftarrow$ Rabin-Karp-Matcher(T,P1,d,q)
2. valid-p2-shifts $\leftarrow$ Rabin-Karp-Matcher(T,P2,d,q)
3. **for** shift1 $\in$ valid-p1-shifts
4. **do for** shift2 $\in$ valid-p2-shifts
5. **do if** shift1 + $m_1 \leq$ shift2
6. **then print** start= shift1, length= shift2 + $m_2 -$ shift1

פתרון

נריץ רבין-קארפ פעם אחת עם P1 ופעם שניה עם P2

ונדפיס את הפתרונות שעבורם האינדקס של הפתרון עם P2 יהיה לפחות m_1 יותר מהאינדקס של הפתרון עם P1:

Find-Wildcard-String(T, P1, P2, m_1 , m_2 , d, q)

1. valid-p1-shifts $\leftarrow$ Rabin-Karp-Matcher(T, P1, d, q) 1+2: $\Theta(n) + O((m_1+m_2)(V+\frac{n}{q}))$
2. valid-p2-shifts $\leftarrow$ Rabin-Karp-Matcher(T, P2, d, q)
3. **for** shift1 $\in$ valid-p1-shifts
4. **do for** shift2 $\in$ valid-p2-shifts 5-10: $O(n^2)$
5. **do if** shift1 + $m_1 \leq$ shift2
6. **then print** start= shift1, length= shift2 + m_2 – shift1

סכ"ה עלות: $O(n^2)$

הוכחת נכונות

בשלב 1-2 נמצא את כל המופעים של P1 ו-P2 ע"י אלגוריתם רבין-קארפ שנכונותו הוכחה.
P2 נדרש להופיע אחרי P1 לפי ההגדרה של הפלט המבוקש ואת זאת מבטיחים בשלב 5 :
מצד אחד, מדפיסים רק מקרים בהם $shift1 + m_1 \leq shift2$

מצד שני, אכן נדפיס את כל המופעים הנכונים, מאחר והערכים ב- valid-p1-shifts וב- valid-p2-shifts הם סדרות עולות ממש ולכל התאמה של תבנית P1 בודקים את כל ההתאמות האפשריות של תבנית P2 וכל התאמה כזו נבדקת פעם אחת.

Find-Wildcard-String(T, P1, P2, m_1 , m_2 , d, q)

1. valid-p1-shifts $\leftarrow$ Rabin-Karp-Matcher(T, P1, d, q)
2. valid-p2-shifts $\leftarrow$ Rabin-Karp-Matcher(T, P2, d, q)
3. **for** shift1 $\in$ valid-p1-shifts
4. **do for** shift2 $\in$ valid-p2-shifts
5. **do if** shift1 + $m_1 \leq$ shift2
6. **then print** start= shift1, length= shift2 + m_2 – shift1

תרגיל 2

נגדיר את בעיית ההתאמה ההופכית:

קלט: $T=t_1...t_n$ ותבנית $P=p_1...p_m$

פלט: כל המיקומים ב-T אשר עבורם מתקיים: $\forall j=1,...,m \quad t_{i+j} \neq p_j$

דוגמה

עבור הקלט $T=5,2,12,6,7$ ו- $P=12,7,5$ יש התאמה בהיסט 0 בטקסט משום ש:

$12 = 12 \text{ and } 7 \neq 6 \text{ and } 5 \neq 7$ משום ש: $12 \neq 5 \text{ and } 7 \neq 2 \text{ and } 5 \neq 12$

התרגיל: כאשר הא"ב מוגבל להיות 0 או 1 בלבד – כתבו אלגוריתם מהיר הפותר את הבעיה. הוכיחו נכונות ונתחו סיבוכיות.

$T = 101101110$

100

$s=1$

דוגמא:

פתרון

הרעיון: נהפוך את ספרותיה של התבנית P . כלומר, כל 0 נהפוך ל-1 ולהיפך.

נריץ אלג' רבין-קארפ על הטקסט T ועל התבנית P לאחר היפוך ספרותיה.

עלות: עלות רבין קרפ

נכונות:

אם רבין-קארפ מוצא התאמה בהיסט s , כלומר מאינדקס $s+m$ יש התאמה אות-אות לתבנית ההפוכה,

אזי אם נהפוך בחזרה את האותיות של התבנית נקבל אי התאמה אות-אות, כנדרש.

תרגיל 3

בהנתן מחרוזות T ו- T'
כתוב אלגוריתם אשר קובע אם מחרוזת T היא סיבוב מעגלי של מחרוזת T' .

הגדרה: מחרוזת T' היא סיבוב מעגלי של מחרוזת $T = t_1 t_2 \dots t_n$ אם קיים $1 \leq i \leq n$ כך שמתקיים:
$$T' = t_i t_{i+1} \dots t_n t_1 t_2 \dots t_{i-1}$$

לדוגמה:

$T = \text{mommy}$

$T' = \text{mymom} \rightarrow T$ הינה סיבוב מעגלי של T'

פתרון

נשים לב ש' T היא סיבוב מעגלי של T אם ורק אם T' מופיעה במחרוזת $T \cdot T$ ובתנאי ששתי המחרוזות T ו- T' באותו האורך.

$$T' = t_i \dots t_n t_1 \dots t_{i-1}$$

$$T \cdot T = t_1 \dots t_i \dots t_n t_1 \dots t_{i-1} t_i \dots t_n$$

פתרון

נשים לב ש' T היא סיבוב מעגלי של T אם ורק אם T' מופיעה במחרוזת $T \cdot T$ ובתנאי ששתי המחרוזות T ו- T' באותו האורך.

$$T' = t_i \dots t_n t_1 \dots t_{i-1}$$

$$T \cdot T = t_1 \dots t_i \dots t_n t_1 \dots t_{i-1} t_i \dots t_n$$

Circular-Rotation-String-Match(T, T', d, q)

1. **if** length(T) != length(T')
2. **then return** False
3. $TT \leftarrow T \cdot T$
4. $\text{valid-shifts} \leftarrow \text{Rabin-Karp-Matcher}(TT, T', d, q)$
5. **if** length(valid-shifts) > 0
6. **then return** True
7. **else return** False

פתרון

נשים לב ש' T היא סיבוב מעגלי של T אם ורק אם T' מופיעה במחרוזת $T \cdot T$ ובתנאי ששתי המחרוזות T ו- T' באותו האורך.

$$T' = t_i \dots t_n t_1 \dots t_{i-1}$$

$$T \cdot T = t_1 \dots t_i \dots t_n t_1 \dots t_{i-1} t_i \dots t_n$$

Circular-Rotation-String-Match(T, T', d, q)

1. **if** length(T) != length(T')
2. **then return** False
3. $TT \leftarrow T \cdot T$
4. $\text{valid-shifts} \leftarrow \text{Rabin-Karp-Matcher}(TT, T', d, q)$
5. **if** length(valid-shifts) > 0
6. **then return** True
7. **else return** False

עלות: עלות רבין קרפ

פתרון

נשים לב ש' T היא סיבוב מעגלי של T אם ורק אם T' מופיעה במחרוזת $T \cdot T$ ובתנאי ששתי המחרוזות T ו- T' באותו האורך.

נכונות:

על פי הגדרה, מחרוזת T' תהיה סיבוב מעגלי של מחרוזת T אם ורק אם קיים $1 \leq i \leq n$ כך שמתקיים:

$$T' = t_i t_{i+1} \dots t_n t_1 t_2 \dots t_{i-1}$$

במידה ואורכי המחרוזות אינו זהה, ההגדרה לא תתקיים ולכן האלגוריתם יחזיר False.

באמצעות שרשור מחרוזת T לעצמה, נקבל מחרוזת חדשה TT אשר כל תת מחרוזת באורך n והיסט s יהווה סיבוב מעגלי של T (ההגדרה תתקיים עבור כל היסט $1 \leq i \leq n$).

אם נמצא התאמה בין מחרוזת T' לתת מחרוזת באורך n והיסט s במחרוזת TT , נקבל ש- T' היא מחרוזת באורך n והיסט s שהיא סיבוב מעגלי של T והאלגוריתם יחזיר True.

אם לא נמצא התאמה בין מחרוזת T' לתת מחרוזת באורך n במחרוזת TT , המחרוזת T' אינה סיבוב מעגלי של T (לא קיים היסט כך שההגדרה מתקיימת) ולכן האלגוריתם יחזיר False. נכונות מציאת ההתאמה מסתמכת על נכונות רבין-קרפ.

$$T' = t_i \dots t_n t_1 \dots t_{i-1}$$

$$T \cdot T = t_1 \dots t_i \dots t_n t_1 \dots t_{i-1} t_i \dots t_n$$

Circular-Rotation-String-Match(T, T', d, q)

1. **if** length(T) != length(T')
2. **then return** False
3. $TT \leftarrow T \cdot T$
4. $\text{valid-shifts} \leftarrow \text{Rabin-Karp-Matcher}(TT, T', d, q)$
5. **if** length(valid-shifts) > 0
6. **then return** True
7. **else return** False